CEUB

Ciências da Computação

Isadora Rodrigues de Freitas Miranda

TaskAnalyzer

Entrega Inicial

Brasília

2026

Sumário

1. Identificação....3
1.1 Link Apresentação....3
2. Introdução....4
2.2. Propósito....4
3. Objetivo Geral....5
4. Executável....6
4.1 Entradas(inputs)....6
4.2 Saídas (Outputs)....6
5. Requisitos....8
5.1 Requisitos Funcionais....8
5.1.1 Entrada e Estruturação de Dados....8
5.1.2 Processamento e Regras de Filtragem....8
5.1.3 Cálculo de Métricas de Tempo e Eficiência....8
5.1.4. Análise e Saída de Dados....9
5.2 Requisitos Não Funcionais....9
5.2.1. Arquitetura e Tecnologia....9
5.2.2. Qualidade e Padrões de Código....9
5.2.3. Resiliência e Confiabilidade....9
5.2.4. Garantia de Qualidade (QA)....10
5.3 Regras de Negócio....10
6. Cenários de Aceite....12
6.1 Cenário 1: Sucesso – Cálculo Correto das Métricas de Produtividade....12
6.2 Cenário 2: Exceção/Erro – Tratamento de Datas Inválidas....12
6.3 Cenário 3: Exceção/Erro – Prevenção de Divisão por Zero....12
7. Test Harness....14
8. Diretrizes Arquiteturais (Obrigatórias)....15
8.1 Proibições Explícitas....15
9. Arquitetura do Repositório....17
9.1 . Árvore....17
9.2. Plano de Homologação....18

1. Identificação

Projeto: TaskAnalyzer

Documento: Especificação SSD TaskAnalyzer - Isadora Rodrigues

Responsável: Isadora Rodrigues de Freitas Miranda

Curso: Ciências da Computação - Asa Norte/Noturno

E-mail: isalala188@sempreceub.com

RA: 22450305

Data de Execução: 09/09/2026

Versão: 1.0

1.1 Link Apresentação

2. Introdução

O presente documento consolida a Entrega Inicial (Fase 1) do projeto do módulo avançado TaskAnalyzer, desenvolvido no âmbito da disciplina do Bootcamp III. Esta etapa marca a imersão na Engenharia de Software contemporânea com o uso de Spec-Driven Development (SDD) integrado à governança avançada de Inteligência Artificial (AI Harness).

2.2. Propósito

O propósito deste documento se resume em quatro pilares principais:

- **Definir o contrato executável de interface**: Estabelecer formalmente as entradas, saídas, tipos de dados e restrições de negócio que o código deverá seguir estritamente na próxima fase.
- **Escopo de atuação**: Atuar como a base técnica e o contrato orientado por especificações (*Spec-Driven Development* - SDD) que guiará a geração automatizada de código por assistentes de Inteligência Artificial.
- **Garantir alinhamento, consistência e qualidade**: Fornecer regras rígidas de governança de contexto (CONTEXT_RULES) e diretrizes para orientar os agentes de IA, evitando ambiguidades e desvios.
- **Planejar a validação objetiva (Test Harness)**: Traduzir cenários de aceite em formato *Dado-Quando-Então* para fundamentar a futura implementação de testes automatizados (pytest), assegurando a homologação correta do código gerado.

3. Objetivo Geral

Desenvolver e documentar uma plataforma de inteligência analítica voltada ao processamento de métricas de produtividade, denominada TaskAnalyzer. O sistema deve ser capaz de analisar tarefas executadas, gerando métricas como tempo médio de conclusão por tipo e prioridade da tarefa, identificar atrasos e gargalos, gerar indicadores consolidados por prioridade e fornecer insumos acionáveis para a tomada de decisão, visando garantir a melhoria contínua e a entrega contínua. Adicionalmente, todo o processo de engenharia deve ser estruturado sob o paradigma do Spec-Driven Development (SDD) e governança de Inteligência Artificial (AI Harness), capacitando o desenvolvedor a atuar como arquiteto de especificações, curador de contexto e homologador crítico no uso de agentes de IA.

4. Executável

Esta seção define o contrato executável de interface da aplicação, especificando rigorosamente a estrutura de dados de entrada que o sistema consome e o formato das saídas e métricas geradas, ressaltando que futuras alterações poderão ser incorporadas conforme a evolução e as necessidades do projeto.

4.1 Entradas(inputs)

O sistema deve receber como entrada uma estrutura de dados contendo uma lista de registros de tarefas. Cada item (tarefa) deve possuir obrigatoriamente os seguintes campos e tipos:

- **id** (str ou int): Identificador único da tarefa.
- **titulo** (str): Nome ou descrição resumida da tarefa.
- **tipo** (str): Categoria ou classificação operacional da tarefa (ex: "Desenvolvimento", "Correção", "Revisão").
- **prioridade** (str): Nível de prioridade atribuído, limitado estritamente a "Alta", "Média" ou "Baixa".
- **data_criacao** (datetime ou str formatado): Data e hora em que a tarefa foi originada no sistema.
- **data_prazo** (datetime ou str formatado): Data e hora limite acordada para a conclusão da tarefa.
- **data_conclusao** (datetime, str formatado ou None): Data e hora em que a tarefa foi efetivamente finalizada (deve ser None ou ausente caso a tarefa ainda esteja pendente).
- **status** (str): Estado atual do ciclo de vida da tarefa (ex: "concluida", "pendente", "cancelada").

4.2 Saídas (Outputs)

O sistema processa o conjunto de entradas e retorna um objeto estruturado (como um dicionário em Python) contendo as seguintes métricas consolidadas:

- **Tempo Médio de Conclusão Global** (float): A média aritmética do tempo de ciclo (em horas ou dias) de todas as tarefas concluídas.
- **Tempo Médio Segmentado por Prioridade** (dict): Indicadores de tempo médio isolados para cada categoria de prioridade ("Alta", "Média", "Baixa").
- **Tempo Médio Segmentado por Tipo** (dict): Indicadores de tempo médio agrupados por tipo/categoria de tarefa.
- **Taxa de Atraso Global** (float): Percentual representativo de tarefas entregues com a data de conclusão posterior ao prazo estimado em relação ao total de concluídas.
- **Média de Atraso** (float): Quantidade média de tempo (dias/horas) excedida pelas tarefas que estouraram o prazo limite.
- **Contagem de Tarefas por Prioridade** (dict): O volume absoluto de tarefas processadas e segmentadas por cada nível de prioridade.
- **Indicadores de Gargalos** (list ou dict): Classificação dos segmentos que apresentam os maiores desvios de tempo ou maiores taxas de atraso, servindo de base para a tomada de decisão gerencial.

5. Requisitos

5.1 Requisitos Funcionais

5.1.1 Entrada e Estruturação de Dados

- **RF01 - Recepção de Dados:** O sistema deve receber um conjunto de dados brutos de tarefas.
- **RF02 - Estrutura Mínima da Tarefa:** Cada tarefa deve conter, no mínimo: ID, título, tipo, nível de prioridade, data de criação, prazo estimado e status atual.
- **RF03 - Validação de Data de Conclusão:** Para tarefas com status "concluída", a data de conclusão efetiva é obrigatória.

5.1.2 Processamento e Regras de Filtragem

- **RF04 - Filtro de Tarefas Efetivadas:** O sistema deve isolar e processar métricas temporais *apenas* sobre tarefas com status finalizado/concluído.
- **RF05 - Validação Cronológica:** O sistema deve verificar a coerência das datas (a data de conclusão não pode ser anterior à data de criação).

5.1.3 Cálculo de Métricas de Tempo e Eficiência

- **RF06 - Tempo Médio Geral:** O sistema deve calcular o tempo médio de ciclo de vida das tarefas concluídas (da criação à conclusão).
- **RF07 - Tempo Médio por Segmento:** O sistema deve segmentar o cálculo do tempo médio por Tipo de Tarefa e por Nível de Prioridade.
- **RF08 - Taxa de Atraso Global:** O sistema deve calcular a porcentagem de tarefas entregues após o prazo estimado.
- **RF09 - Média de Dias em Atraso:** O sistema deve quantificar a média de tempo (em dias/horas) em que as tarefas excederam o prazo estipulado.

5.1.4. Análise e Saída de Dados

- **RF10 - Mapeamento de Gargalos:** O sistema deve classificar e ordenar quais tipos de tarefas ou prioridades possuem os maiores desvios de tempo médio.
- **RF11 - Agregação por Prioridade:** O sistema deve gerar um consolidado com a contagem total de tarefas por nível de prioridade (alta, média, baixa).
- **RF12 - Relatório Estruturado:** O sistema deve retornar as métricas processadas em um formato estruturado (ex: Dicionário Python ou JSON) para facilitar a integração e a tomada de decisão.

5.2 Requisitos Não Funcionais

5.2.1. Arquitetura e Tecnologia

- **RNF01 - Linguagem Base:** A aplicação deve ser implementada em Python 3.11 ou superior.
- **RNF02 - Tipagem Estrita:** Uso obrigatório de *type hints* para todos os métodos, parâmetros e retornos.

5.2.2. Qualidade e Padrões de Código

- **RNF03 - Conformidade de Estilo:** O código deve estar 100% alinhado ao guia de estilo PEP 8.
- **RNF04 - Responsabilidade Única (SRP):** As funções devem ser granulares, altamente coesas e executar um único propósito lógico.
- **RNF05 - Documentação Integrada:** Implementação obrigatória de *Google Style Docstrings* para todas as classes e funções.

5.2.3. Resiliência e Confiabilidade

- **RNF06 - Tratamento Específico de Erros:** Exceções devem ser capturadas especificamente (ex: prevenção de divisão por zero, erros de tipagem) sem interrupção genérica do sistema.
- **RNF07 - Observabilidade (Logs):** Todos os fluxos, métricas principais e erros devem ser registrados via biblioteca padrão de logging.

5.2.4. Garantia de Qualidade (QA)

- **RNF08 - Automação de Testes:** O comportamento das lógicas de negócio deve ser coberto e validado utilizando o framework pytest.
- **RNF09 - Cobertura Contratual:** O código deve passar por testes rigorosos de cenários de sucesso e falha (exceções) baseados no formato Dado-Quando-Então do contrato de aceite.

5.3 Regras de Negócio

RN01 - Critério de Inclusão Temporal: Para o cálculo de qualquer métrica temporal (tempo médio de conclusão, dias em atraso), o sistema deve considerar exclusivamente as tarefas que contenham o status de concluídas (concluida), descartando tarefas pendentes ou canceladas.

RN02 - Validação de Consistência Cronológica: A data e hora de conclusão efetiva de uma tarefa nunca podem ser anteriores à sua data e hora de criação. Caso essa inconsistência seja identificada, o sistema deve rejeitar o registro e disparar uma exceção de negócio específica (ValueError ou exceção customizada).

RN03 - Mitigação de Divisão por Zero: O sistema deve implementar validações defensivas preventivas para cenários onde a lista de tarefas concluídas esteja vazia. Ao tentar calcular médias ou taxas proporcionais sem dados válidos, a aplicação deve retornar valores zerados padronizados (0.0) ou tratar a exceção controladamente para evitar interrupções abruptas.

RN04 - Segmentação Obrigatória de Métricas: As métricas de desempenho e tempo médio de conclusão não devem ser apenas consolidadas de forma global, mas obrigatoriamente segmentadas e calculadas por nível de prioridade (Alta, Média, Baixa) e por tipo de tarefa.

RN05 - Apuração de Atrasos: Uma tarefa será categorizada como "entregue com atraso" se a data/hora de conclusão efetiva for estritamente superior à data limite estimada (prazo) cadastrada.

RN06 - Imutabilidade do Contrato de Interface: As assinaturas das funções públicas e a estrutura de dados de entrada aceita pelo módulo de análise não podem ser modificadas por agentes de IA ou scripts secundários sem uma atualização formal prévia do contrato SDD.

6. Cenários de Aceite

6.1 Cenário 1: Sucesso – Cálculo Correto das Métricas de Produtividade

Dado um conjunto de tarefas válidas contendo diferentes prioridades (Alta, Média, Baixa), datas de criação consistentes, datas de conclusão preenchidas e status igual a concluído (concluída),

Quando o módulo analisador de tarefas (task_analyzer.py) for executado com este conjunto de dados,

Então o sistema deve retornar com sucesso as métricas consolidadas, contemplando:

1) O tempo médio correto de conclusão segmentado por prioridade e o geral.
2) A taxa de atraso precisa (percentual de tarefas finalizadas após a data limite estimada).
3) A contagem total exata de tarefas processadas por nível de prioridade.

6.2 Cenário 2: Exceção/Erro – Tratamento de Datas Inválidas

Dado um conjunto de dados de entrada onde uma ou mais tarefas possuem a data de conclusão anterior à data de criação (inconsistência cronológica),

Quando o módulo analisador for acionado para processar essas informações,

Então o sistema deve interceptar a inconsistência e disparar uma exceção específica tratada (ex: ValueError ou exceção customizada de negócio), exibindo uma mensagem clara, descritiva e orientativa ao usuário sem interromper a execução abruptamente por falhas não tratadas.

6.3 Cenário 3: Exceção/Erro – Prevenção de Divisão por Zero

Dado um conjunto de dados de entrada que não contém nenhuma tarefa com o status de concluída (lista vazia de conclusões para o cálculo de médias),

Quando o analisador tentar calcular o tempo médio de conclusão ou a taxa proporcional,

Então o sistema deve identificar o risco de divisão por zero, tratar a condição de forma defensiva e retornar valores zerados padronizados (0.0) ou disparar uma exceção controlada de negócio, garantindo a estabilidade da aplicação.

7. Test Harness

Criação de um ambiente de testes automatizados utilizando o framework **pytest**, estruturado para validar de forma objetiva e sem intervenção humana se o código gerado via IA atende integralmente ao contrato de negócios.

1. **Estrutura de Arquivos:** Os testes residirão na pasta dedicada tests/test_harness.py, isolando a lógica de validação do código de produção (src/task_analyzer.py).
2. Mapeamento de Casos de Teste (pytest):
    - Serão implementadas funções de teste unitário e de integração (ex: test_calcular_metricas_sucesso, test_datas_invalidas_excecao, e test_divisao_por_zero_vazio) que espelham diretamente os cenários de aceite descritos acima.
3. **Uso de Fixtures:** Utilização de *fixtures* do pytest para injetar conjuntos de dados simulados (*mocks* de tarefas com cenários válidos, inválidos e extremos) de maneira repetível e isolada.
4. **Critério de Aceitação da IA:** Na Fase 2, nenhum código gerado pelo agente de IA será considerado aprovado ou *committado* para o repositório principal sem que 100% dos testes do *Test Harness* passem com sucesso na linha de comando (pytest -v).

8. Diretrizes Arquiteturais (Obrigatórias)

1. **Versão Mínima da Linguagem:** O código deve ser desenvolvido estritamente utilizando Python 3.11 ou superior.
2. **Tipagem Estática:** É obrigatório o uso de *type hints* (anotações de tipo) em todas as funções, métodos e assinaturas de classes.
3. **Padrões de Código (PEP 8 & SRP):** O código deve seguir rigorosamente o guia de estilo PEP 8 e o Princípio da Responsabilidade Única (*Single Responsibility Principle - SRP*).
4. **Documentação Padronizada:** Todos os módulos, classes, funções e parâmetros devem conter docstrings formais estruturadas no padrão Google Style.
5. **Coesão e Modularidade:** As funções devem ser pequenas, altamente coesas, independentes e projetadas para um único propósito de negócio.
6. **Tratamento de Exceções:** O sistema deve implementar tratamento de exceções específico (evitando blocos genéricos de captura) acompanhado de mensagens claras e descritivas.
7. **Logs Estruturados:** A rastreabilidade de execução deve ser garantida por meio do uso do módulo padrão logging do Python.
8. **Legibilidade e Nomenclatura:** O código deve priorizar a manutenibilidade, empregando nomes de variáveis, funções e classes descritivos e em conformidade com o domínio do problema.

8.1 Proibições Explícitas

- Bibliotecas externas que não estejam explicitamente autorizadas no escopo do projeto não devem ser utilizadas.
- Os cenários de teste estabelecidos no planejamento inicial não devem ser alterados, removidos ou adaptados.
- A estrutura de pastas e arquivos definida para o repositório não deve ser modificada sem autorização prévia.
- A lógica de persistência de dados em arquivos locais ou bancos de dados não deve ser implementada.

- As assinaturas (nomes e parâmetros) das funções públicas descritas no contrato de interface não devem ser alteradas.
- Nenhum código de nova funcionalidade deve ser gerado sem a respectiva implementação de testes correspondentes.
- A inserção de código duplicado, redundante ou sem necessidade arquitetural não deve ser realizada.
- Comportamentos, regras ou lógicas que não estejam explicitamente detalhadas no contrato de negócio não devem ser assumidos pelo agente de IA.

9. Arquitetura do Repositório

9.1 . Árvore

A estrutura de arquivos e diretórios que será versionada no GitHub na próxima etapa do projeto segue o modelo abaixo:

```
sdd-task-analyzer/
├── .github/
│   └── workflows/
│       └── ci_pipeline.yml       # Pipeline de Integração Contínua (CI/CD - Fase 3)
├── src/
│   ├── __init__.py
│   ├── task_analyzer.py          # Código principal gerado e homologado
│   └── exceptions.py             # Exceções customizadas de negócio
├── tests/
│   ├── __init__.py
│   └── test_harness.py           # Testes automatizados (pytest)
├── specs/
│   └── task_analyzer_spec.md     # Documentação SDD consolidada (Fase 1)
├── .gitignore                    # Arquivos ignorados pelo Git (.pytest_cache, venv, etc.)
```

```
├── Dockerfile              # Configuração para containerização (Fase 3)
├── requirements.txt        # Dependências do projeto (pytest, etc.)
├── CONTEXT_RULES.md        # Regras persistentes de governança para a IA
└── README.md               # Documentação de apresentação e guias do projeto
```

9.2. Plano de Homologação

1. **Validação Funcional:** Execução completa da suíte de testes automatizados (pytest) para certificar que o comportamento atende aos cenários de aceite.
2. **Conformidade Contratual:** Revisão detalhada da aderência do código ao contrato de negócio e às diretrizes do arquivo CONTEXT_RULES.md.
3. **Análise de Qualidade:** Inspeção da legibilidade, adoção de *type hints*, tratamento adequado de erros e aplicação correta do padrão PEP 8.
4. **Verificação de Restrições:** Conferência rigorosa para assegurar que nenhuma das proibições explícitas foi violada pelo assistente de IA.
5. **Testes Manuais de Borda:** Execução de validações complementares em cenários extremos ou atípicos, quando julgado necessário.
6. **Aprovação e Commit:** Somente após a aprovação integral em todas as etapas anteriores, o código será aceito, versionado e *comitado* no repositório.